

COMP2300-COMP6300-ENGN2219

Computer Architecture

Convener: Shoaib Akram

shoaib.akram@anu.edu.au



Australian
National
University

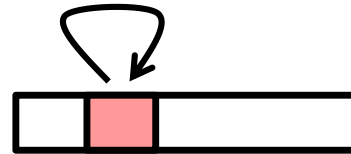
Locality and its Exploitation

Locality

- **Principle of Locality:** Programs tend to use data and instructions with addresses near or equal to those they have used recently

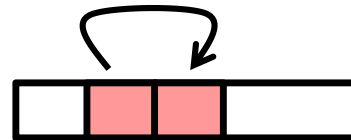
- **Temporal locality:**

- Recently referenced items are likely to be referenced again in the near future



- **Spatial locality:**

- Items with nearby addresses tend to be referenced close together in time



Locality Example

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

■ Data references

- Reference array elements in succession (stride-1 reference pattern).
- Reference variable `sum` each iteration.

Spatial locality

Temporal locality

■ Instruction references

- Reference instructions in sequence.
- Cycle through loop repeatedly.

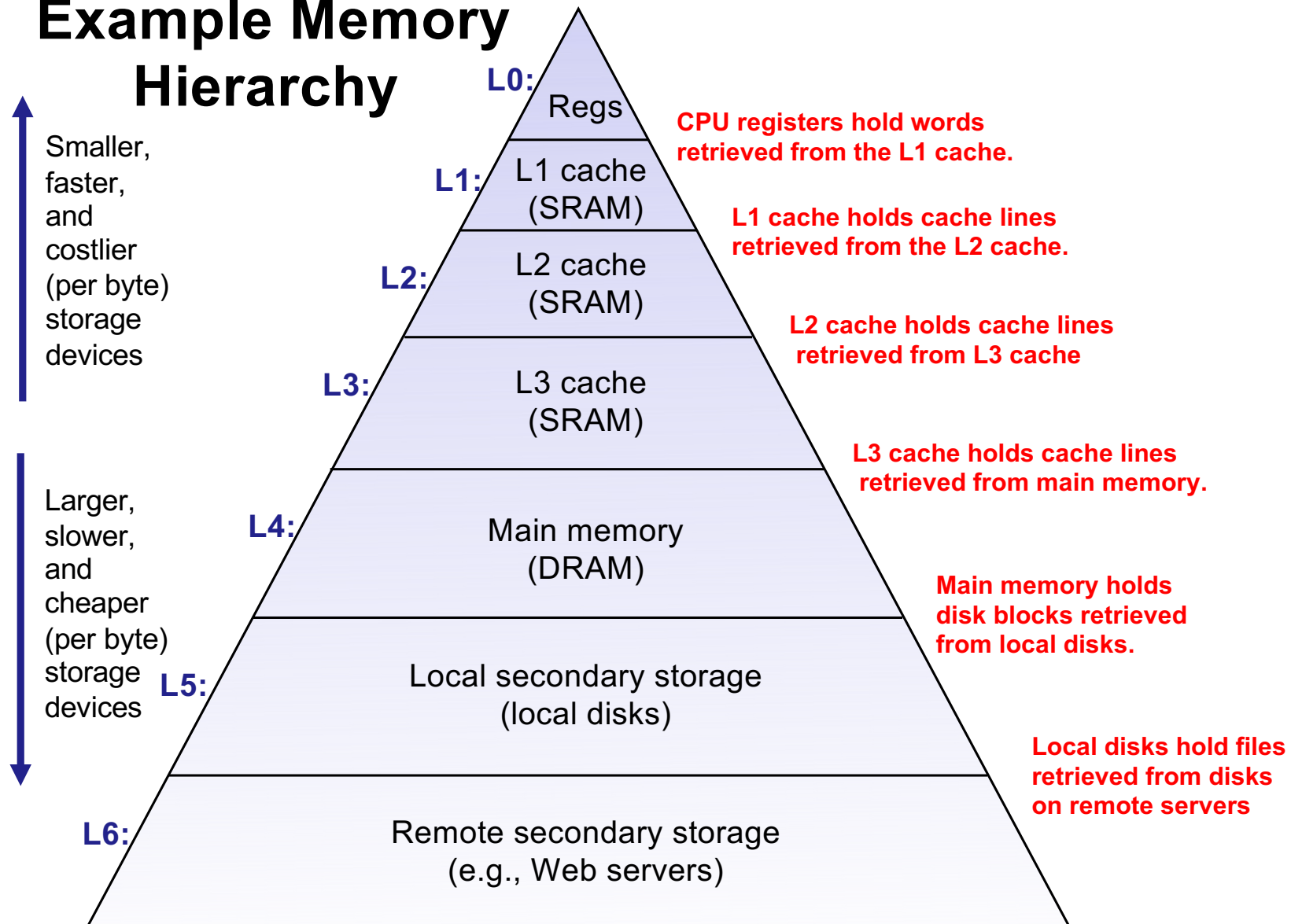
Spatial locality

Temporal locality

Memory Hierarchies

- **Some fundamental and enduring properties of hardware and software:**
 - Fast storage technologies cost more per byte, have less capacity, and require more power (heat!).
 - The gap between CPU and main memory speed is widening.
 - Well-written programs tend to exhibit good locality.
- **These fundamental properties complement each other beautifully.**
- **They suggest an approach for organizing memory and storage systems known as a **memory hierarchy**.**

Example Memory Hierarchy



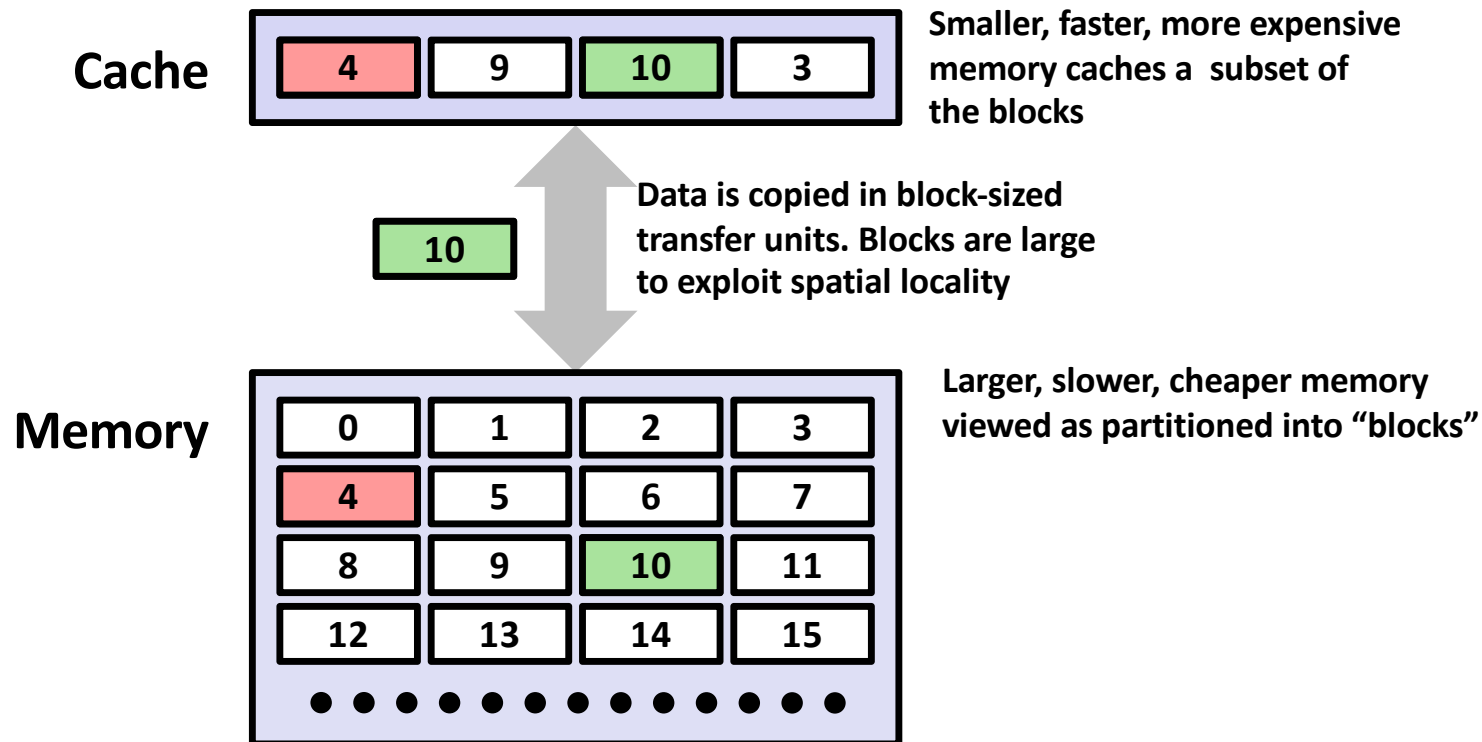
Cache

- **Cache:** A smaller, faster storage device that acts as a staging area for a subset of the data in a larger, slower device.
- **Fundamental idea of a memory hierarchy:**
 - For each k , the faster, smaller device at level k serves as a cache for the larger, slower device at level $k+1$.
- **Why do memory hierarchies work?**
 - Because of locality, programs tend to access the data at level k more often than they access the data at level $k+1$.
 - Thus, the storage at level $k+1$ can be slower, and thus larger and cheaper per bit.
- **Big Idea:** The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

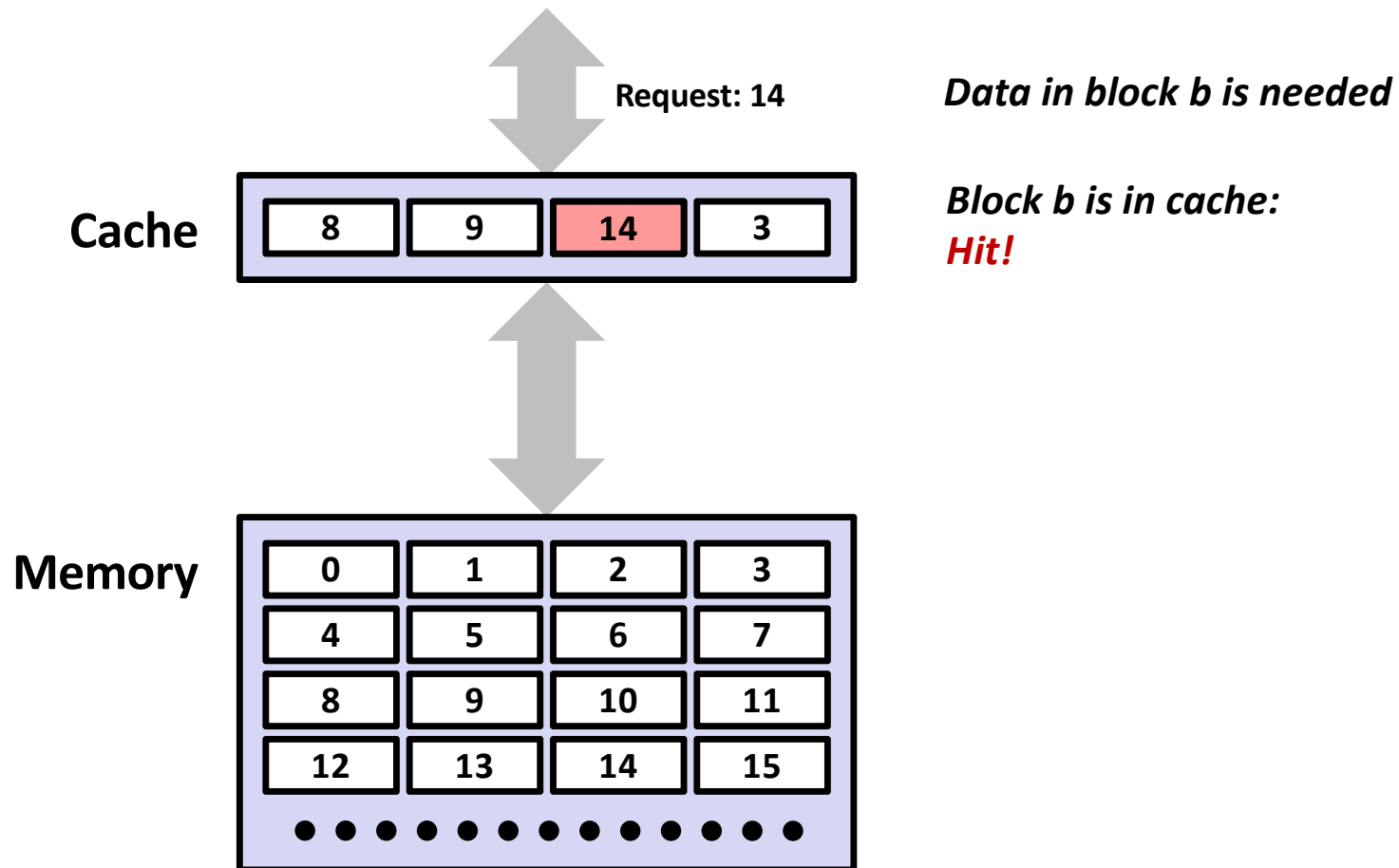
How does Caches Work?

- **Cache memories** are small, fast SRAM-based memories managed automatically in hardware
 - Hold frequently accessed blocks of main memory
 - CPU looks first for data in cache, then in memory
- **Implementation technology across the memory hierarchy**
 - Caches are built out of SRAM cells – using cross-coupled inverters
 - Main memory is built out of DRAM cells – using capacitor/transistor combination
 - Storage device (non-volatile or persistent) is built out of Flash memory cells
- **In high-performance systems, SRAM cache(s) back DRAM memory**

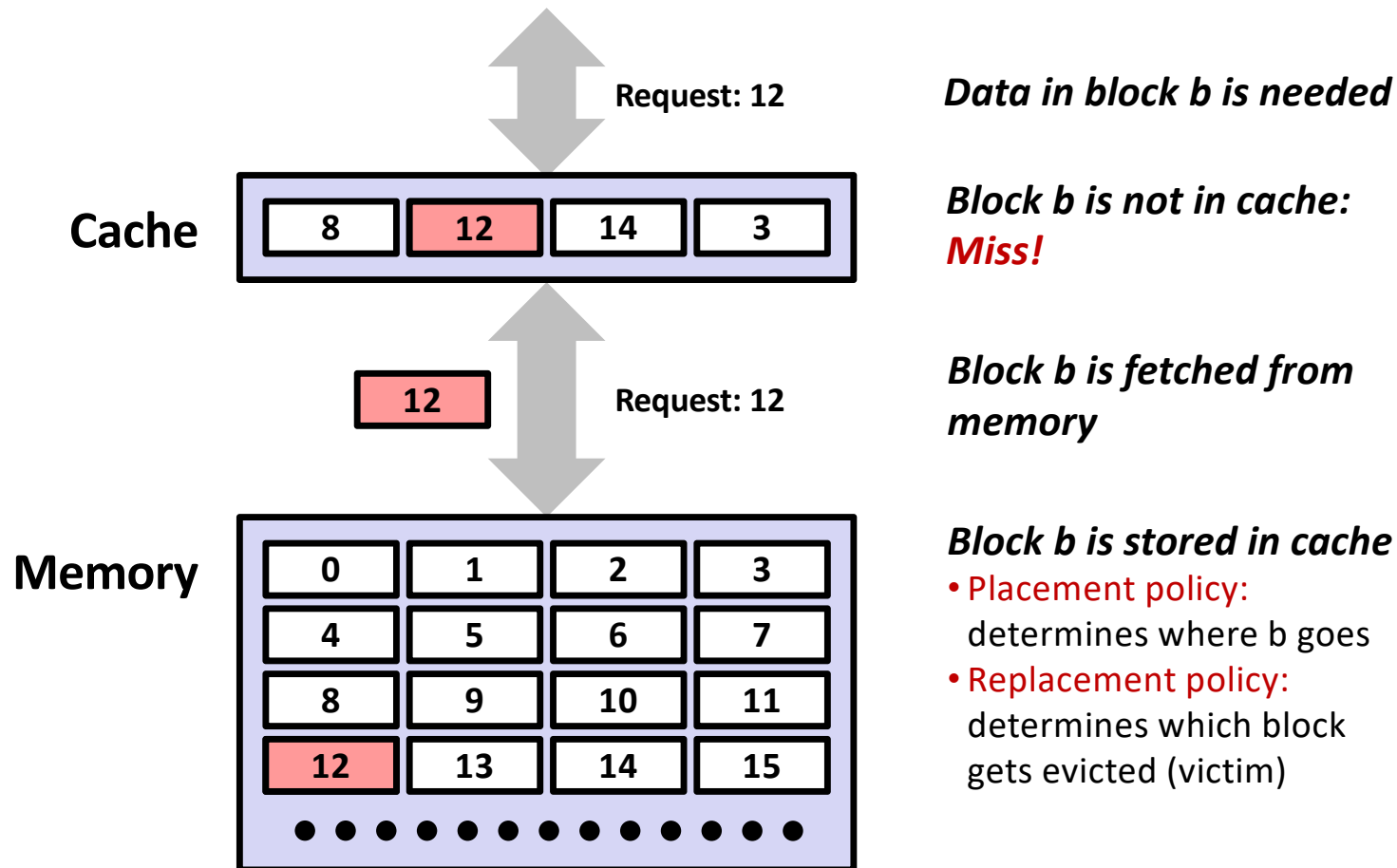
General Cache Concepts



General Cache Concepts: Hit



General Cache Concepts: Miss



Cache Performance: Hit Rate and Miss Rate

■ Miss Rate

- Fraction of memory references not found in cache (misses / accesses)
= $1 - \text{hit rate}$
- Typical numbers (in percentages):
 - 3-10% for L1
 - can be quite small (e.g., $< 1\%$) for L2, depending on size, etc.

■ Hit Time

- Time to deliver a line in the cache to the processor
 - includes time to determine whether the line is in the cache
- Typical numbers:
 - 4 clock cycle for L1
 - 10 clock cycles for L2

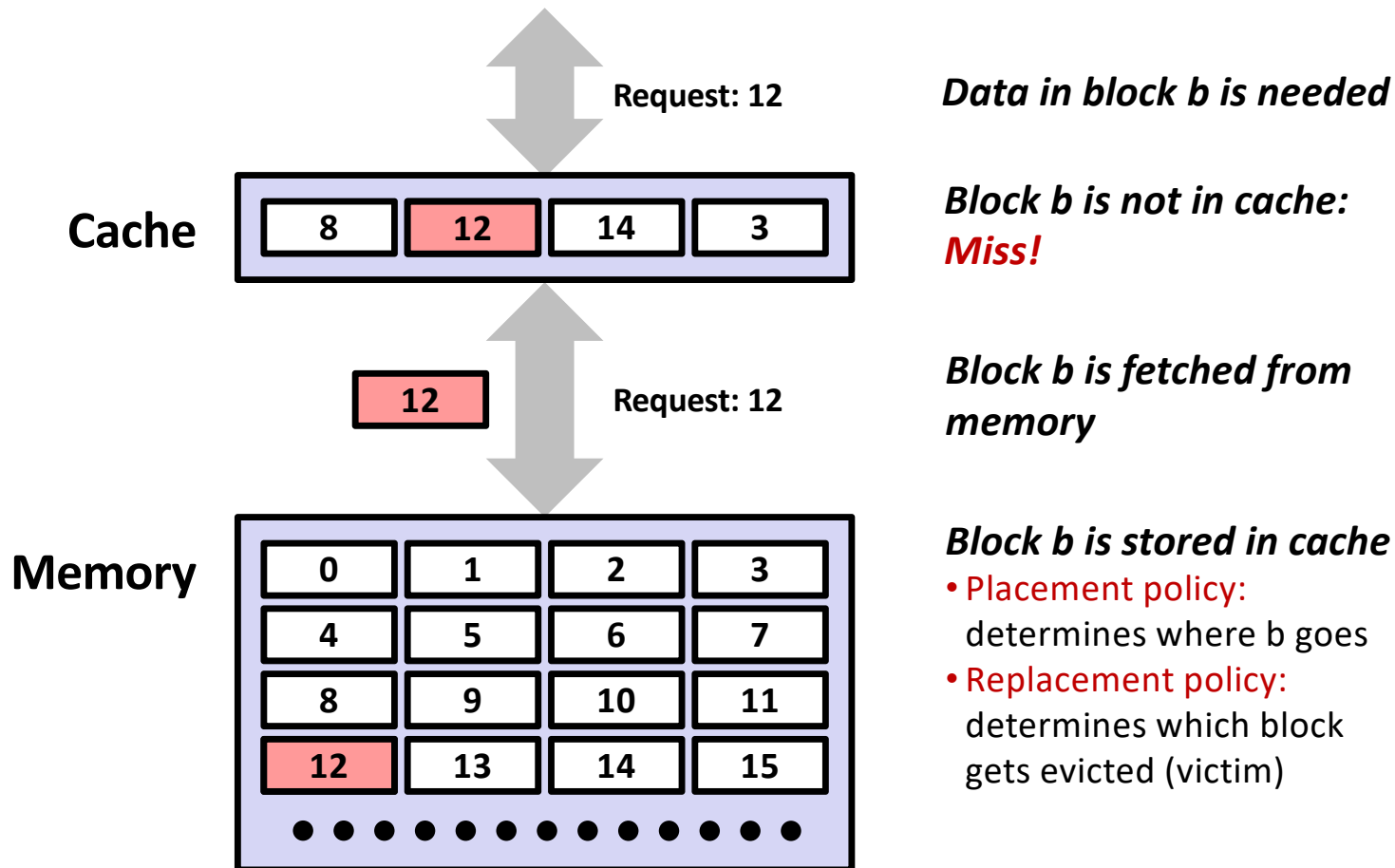
■ Miss Penalty

- Additional time required because of a miss
 - typically 50-200 cycles for main memory (Trend: increasing!)

Let's think about those numbers

- Huge difference between a hit and a miss
 - Could be 100x, if just L1 and main memory
- Would you believe 99% hits is twice as good as 97%?
 - Consider:
cache hit time of 1 cycle
miss penalty of 100 cycles
 - Average access time:
97% hits: $1 \text{ cycle} + 0.03 * 100 \text{ cycles} = 4 \text{ cycles}$
99% hits: $1 \text{ cycle} + 0.01 * 100 \text{ cycles} = 2 \text{ cycles}$
- This is why “miss rate” is used instead of “hit rate”

General Cache Concepts: Replacement Policy

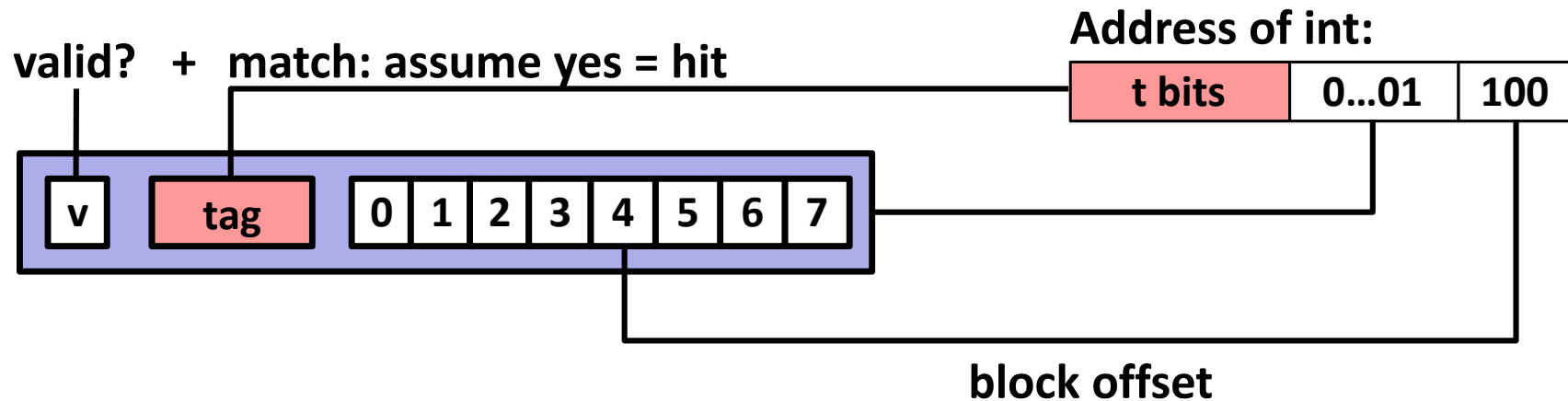


General Cache Concepts: (Re)Placement Policies

- **Cache line: A block of data is associated with a tag (portion of memory address)**
 - Hence, these caches are called associative caches (tag bits are stored along with data)
 - Some bits are used to find the byte within the block, and location to place the block
- **Each block in slower level can be placed in a set of n possible locations in the fast level**
 - n -way set-associative cache
 - More choice regarding placement of blocks (hence, requires more silicon to manufacture)
 - Low miss rate (high hit rate)
- **Each block in slower level can be placed in exactly one location in the fast level**
 - Direct-mapped cache ($n = 1$ or 1-way set-associative)
 - Simple to implement
 - High miss rate due to frequent conflicts (low hit rate)
- **Replacement policy is needed when the cache is full (e.g., least recently used or LRU)**

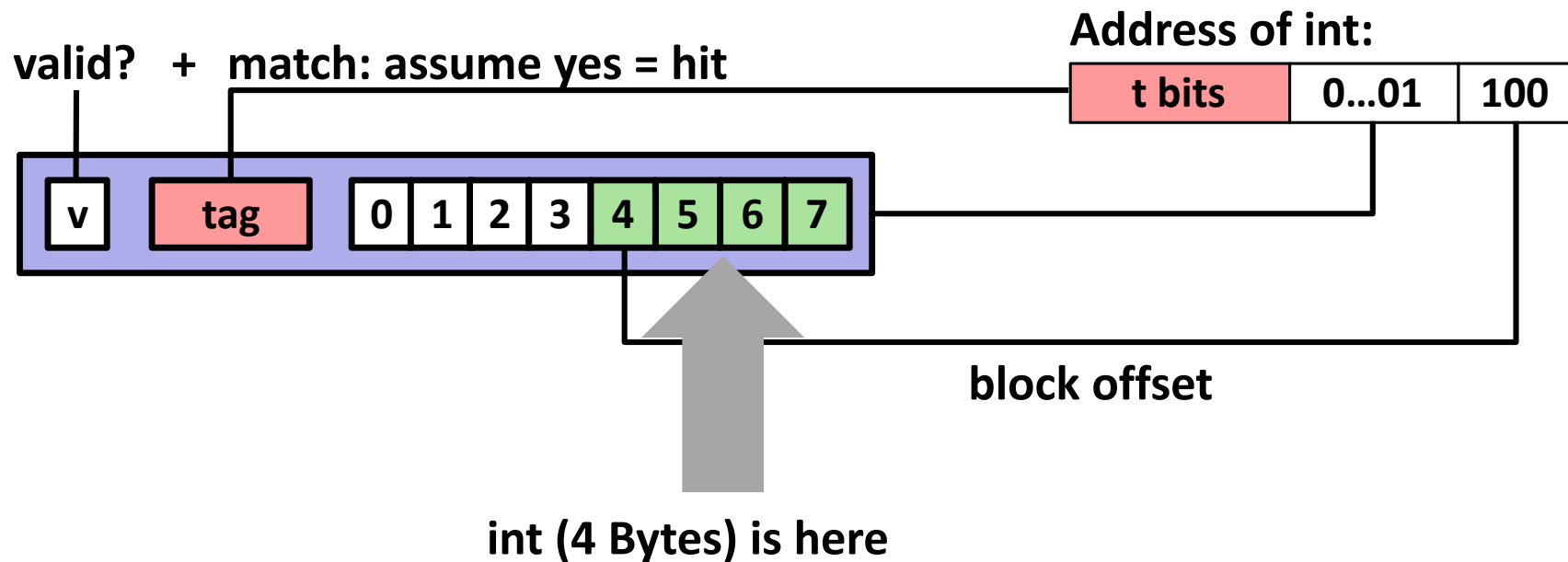
Example: Direct Mapped Cache (n = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



Example: Direct Mapped Cache (n = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



- **If tag doesn't match:** old line is evicted and replaced

Simulation: Direct Mapped Cache (n = 1)

t=1	s=2	b=1
x	xx	x

M=16 bytes (4-bit addresses), B=2 bytes/block,
S=4 sets, n=1 Blocks/set

Address trace (reads, one byte per read):

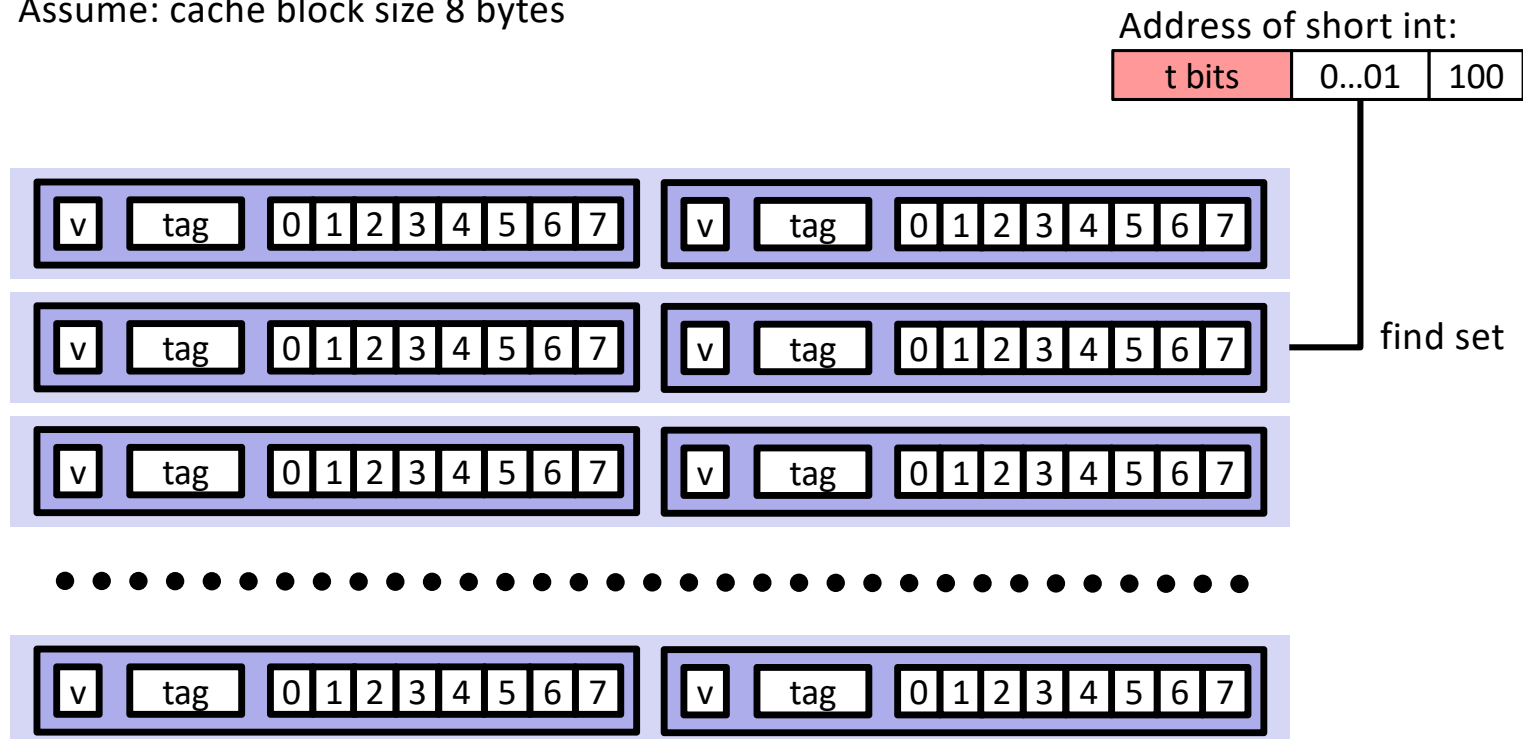
0	[0000] ₂ ,	miss
1	[0001] ₂ ,	hit
7	[0111] ₂ ,	miss
8	[1000] ₂ ,	miss
0	[0000] ₂	miss

	v	Tag	Block
Set 0	1	0	M[0-1]
Set 1			
Set 2			
Set 3	1	0	M[6-7]

Example: 2-way Set Associative Cache (n = 2)

n = 2: Two lines per set

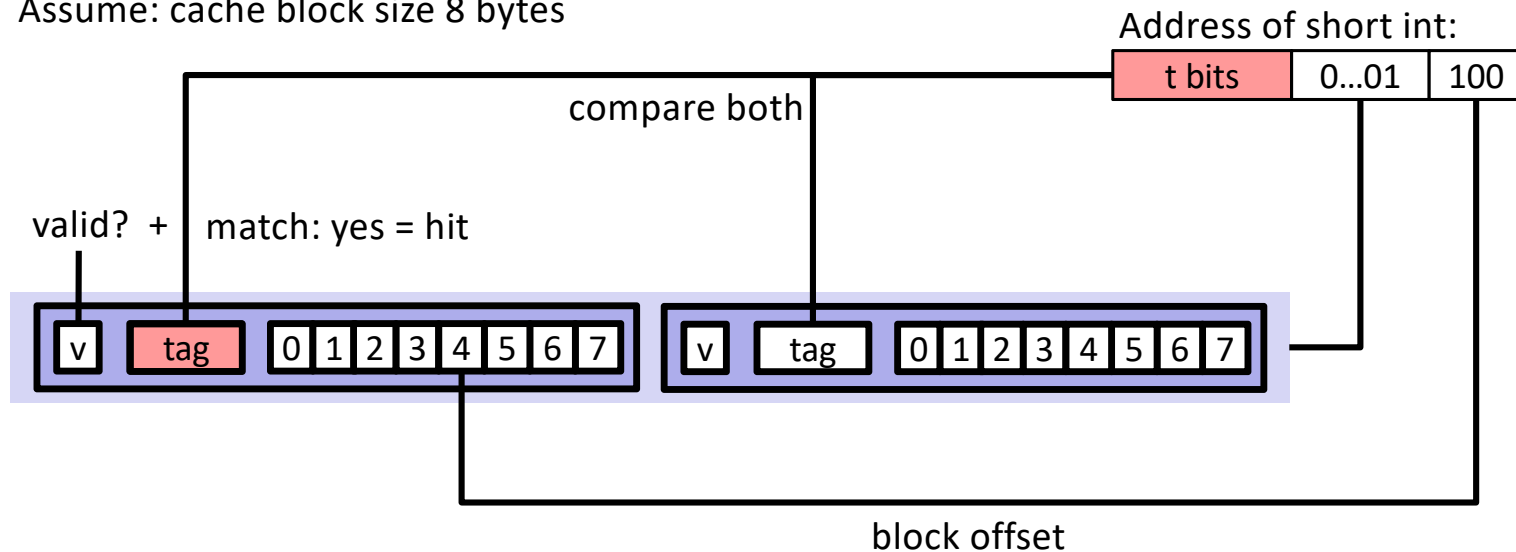
Assume: cache block size 8 bytes



Example: 2-way Set Associative Cache (n = 2)

n = 2: Two lines per set

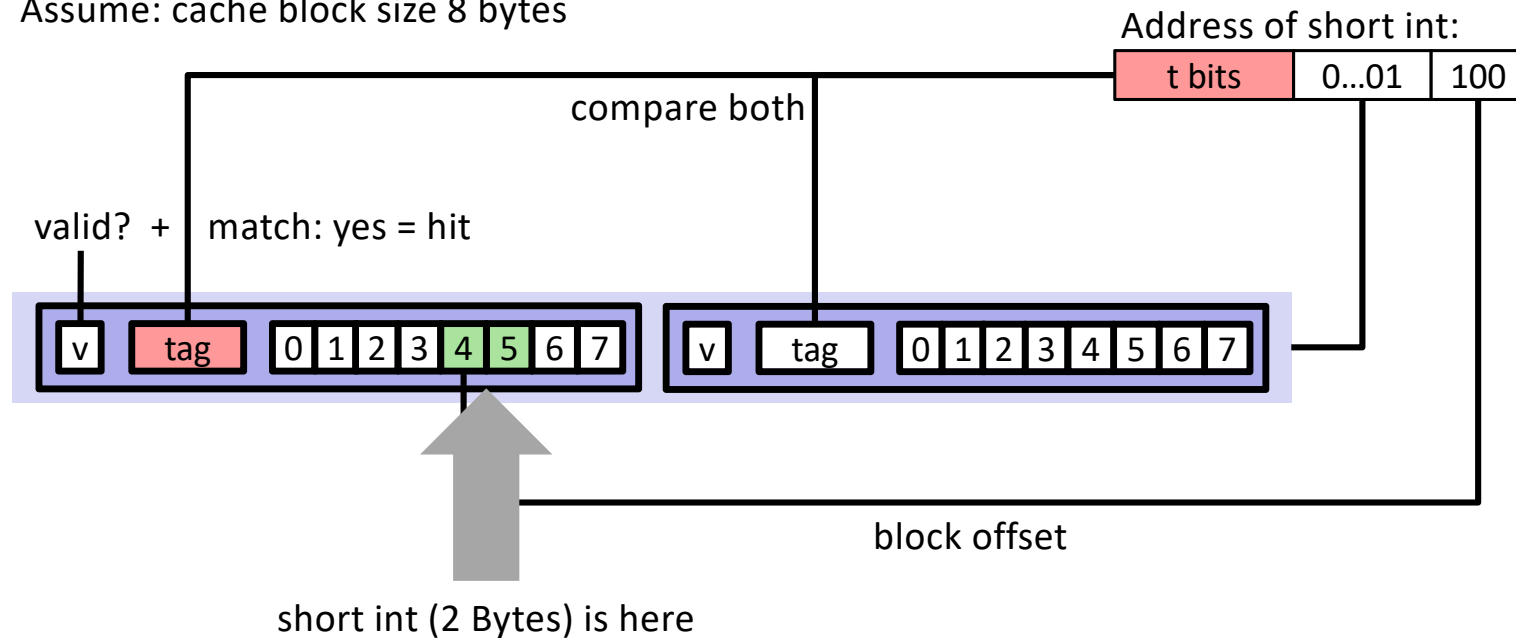
Assume: cache block size 8 bytes



Example: 2-way Set Associative Cache (n = 2)

n = 2: Two lines per set

Assume: cache block size 8 bytes



No match:

- One line in set is selected for eviction and replacement
- Replacement policies: random, least recently used (LRU), ...

Simulation: 2-way Set Associative Cache (n = 2)

t=2	s=1	b=1
xx	x	x

M=16 byte (4-bit addresses), B=2 bytes/block,
S=2 sets, n=2 blocks/set

Address trace (reads, one byte per read):

0	[00 <u>0</u> 0] ₂ ,	miss
1	[00 <u>0</u> 1] ₂ ,	hit
7	[01 <u>1</u> 1] ₂ ,	miss
8	[10 <u>0</u> 0] ₂ ,	miss
0	[00 <u>0</u> 0] ₂	hit

	v	Tag	Block
Set 0	1	00	M[0-1]
	1	10	M[8-9]
Set 1	1	01	M[6-7]
	0		

What about Writes?

■ Multiple copies of data exist:

- L1, L2, L3, Main Memory, Disk

■ What to do on a write-hit?

- **Write-through** (write immediately to memory)
- **Write-back** (defer write to memory until replacement of line)
 - Need a dirty bit (line different from memory or not)

■ What to do on a write-miss?

- **Write-allocate** (load into cache, update line in cache)
 - Good if more writes to the location follow
- **No-write-allocate** (writes straight to memory, does not load into cache)

■ Typical

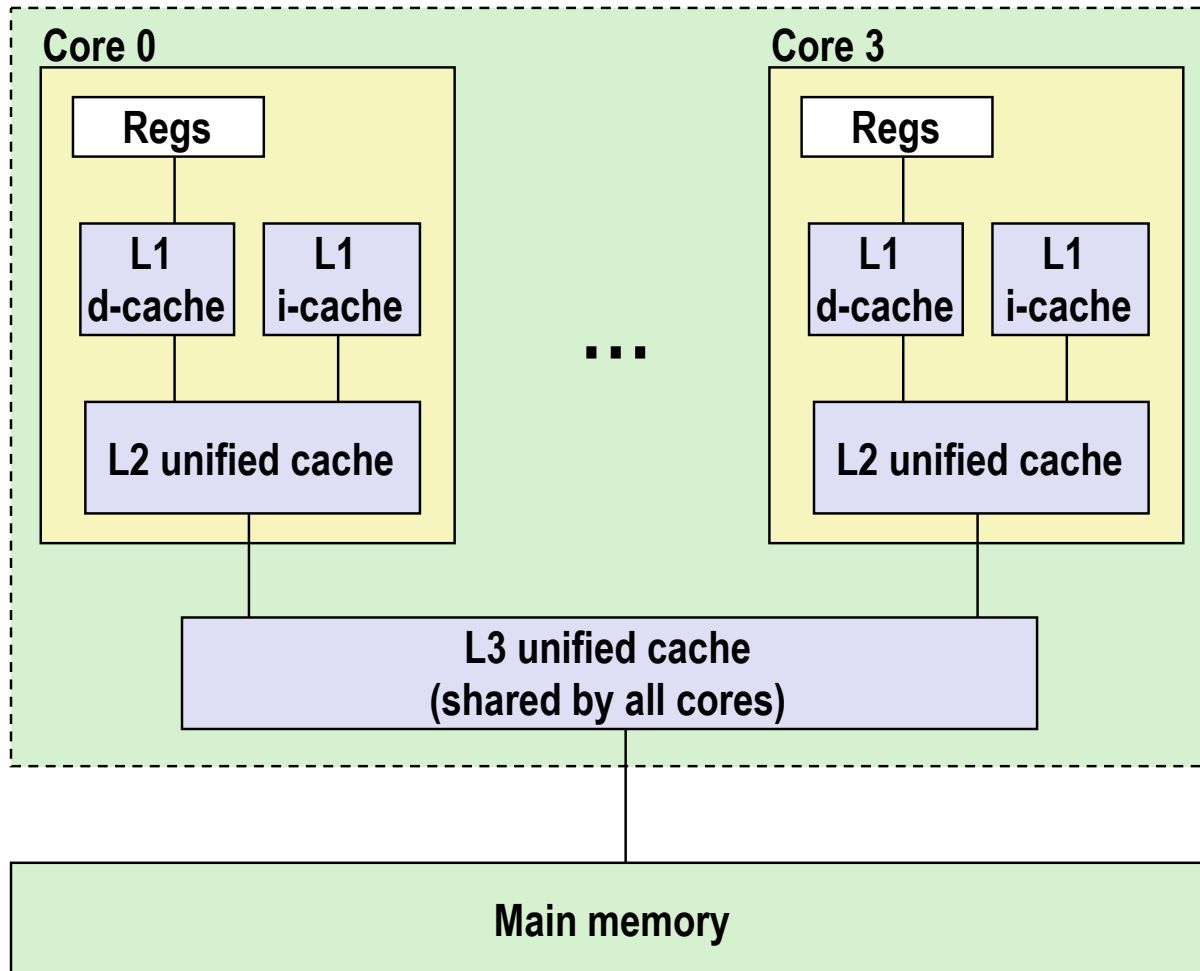
- Write-through + No-write-allocate
- **Write-back + Write-allocate**

Why do we have a cache hierarchy?

- **Smaller is faster (general principle in computer architecture)**
 - Very fast (SRAM) cache closer to the CPU
 - Progressively slower (SRAM) caches at next levels balance cost and performance
 - This results in a cache hierarchy with typical #levels = 3
- Level 1 (L1) cache is 10s of kilobytes with access latency of 1 clock cycle
 - CPU never stalls (waits) for next instruction or piece of data
- Level 2 (L2) is ~10 cycles
- Level 3 is ~30 cycles
- **L1 caches are split into two caches: instruction and data**

Intel Core i7 Cache Hierarchy

Processor package



L1 i-cache and d-cache:

32 KB, 8-way,
Access: 4 cycles

L2 unified cache:

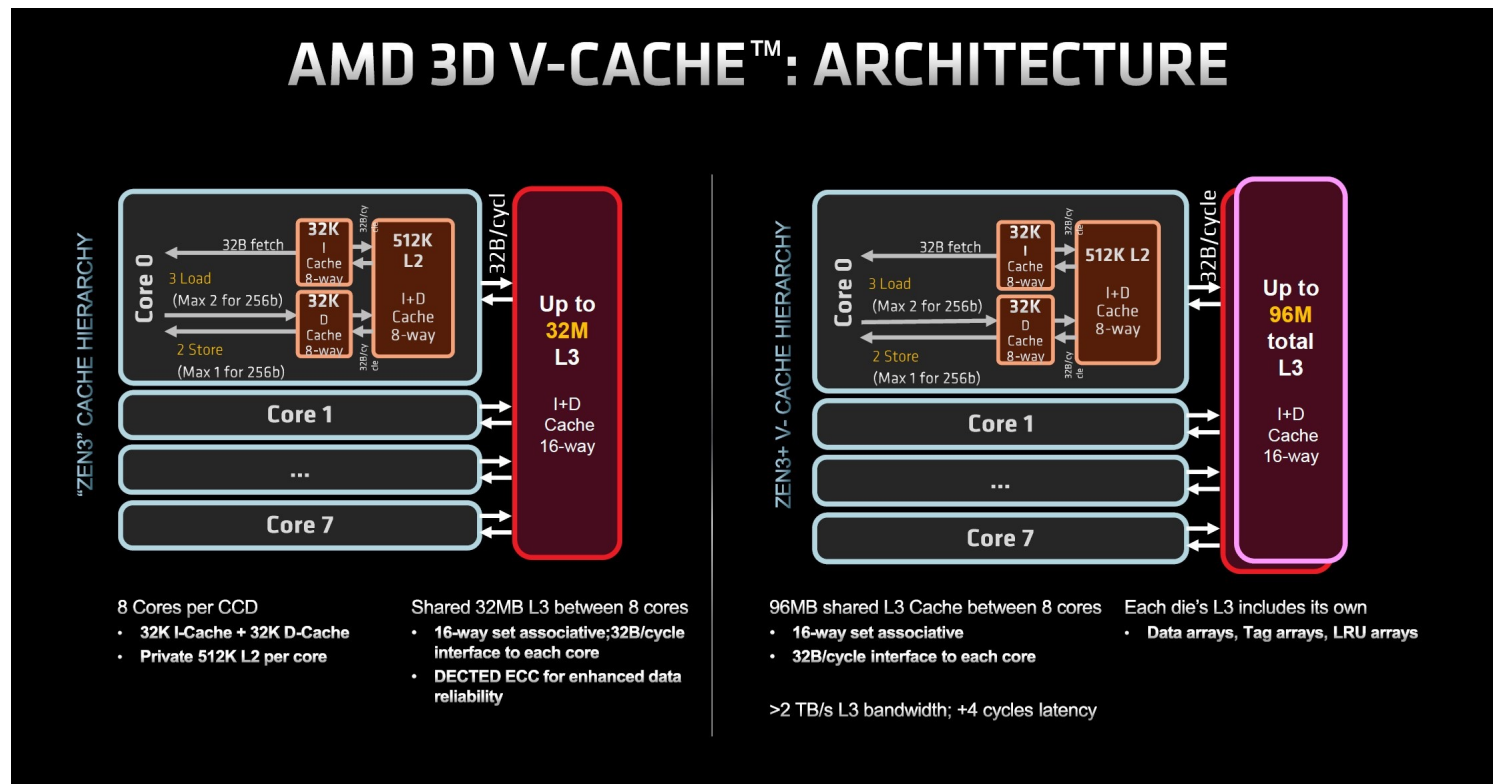
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:

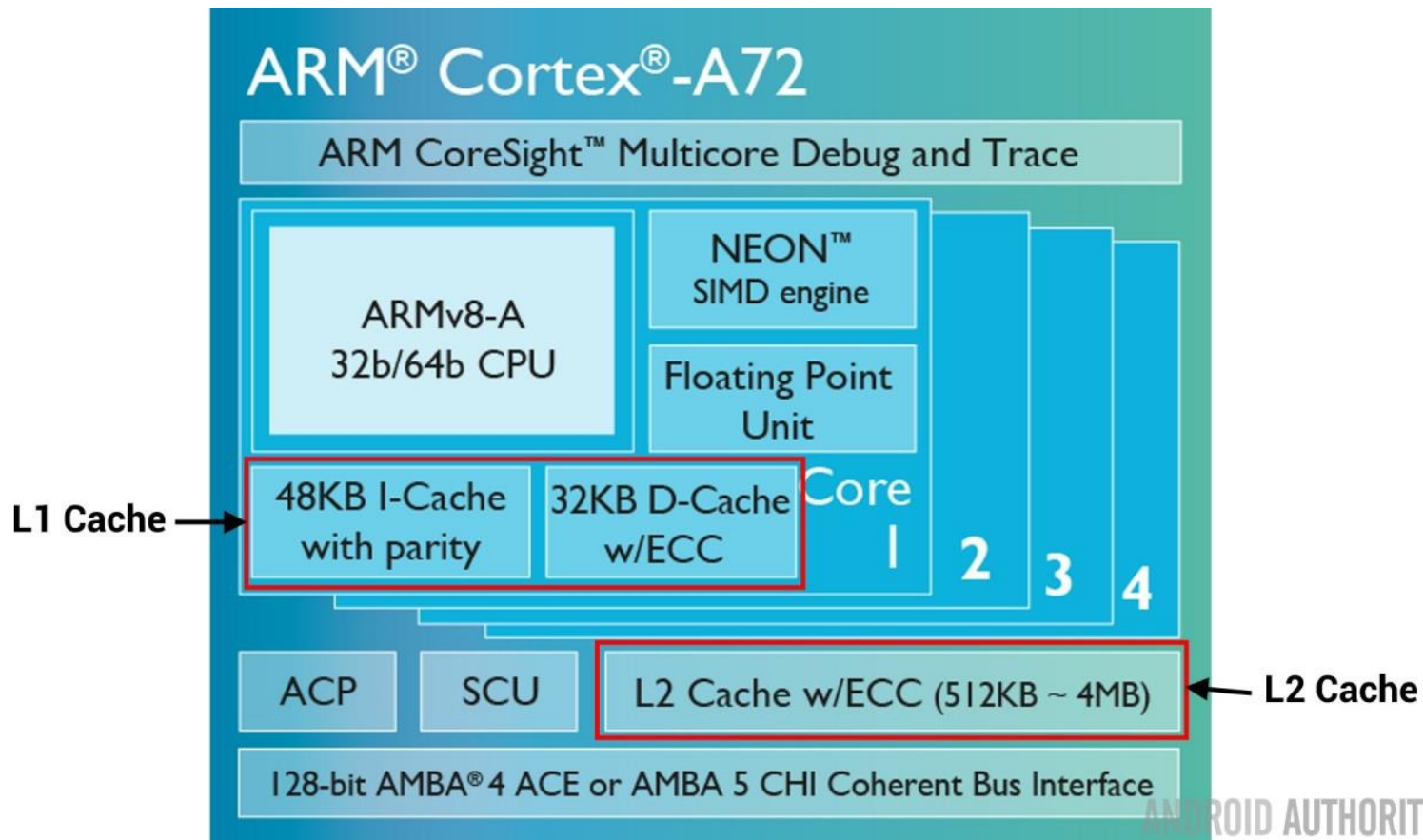
8 MB, 16-way,
Access: 40-75 cycles

Block size: 64 bytes for
all caches.

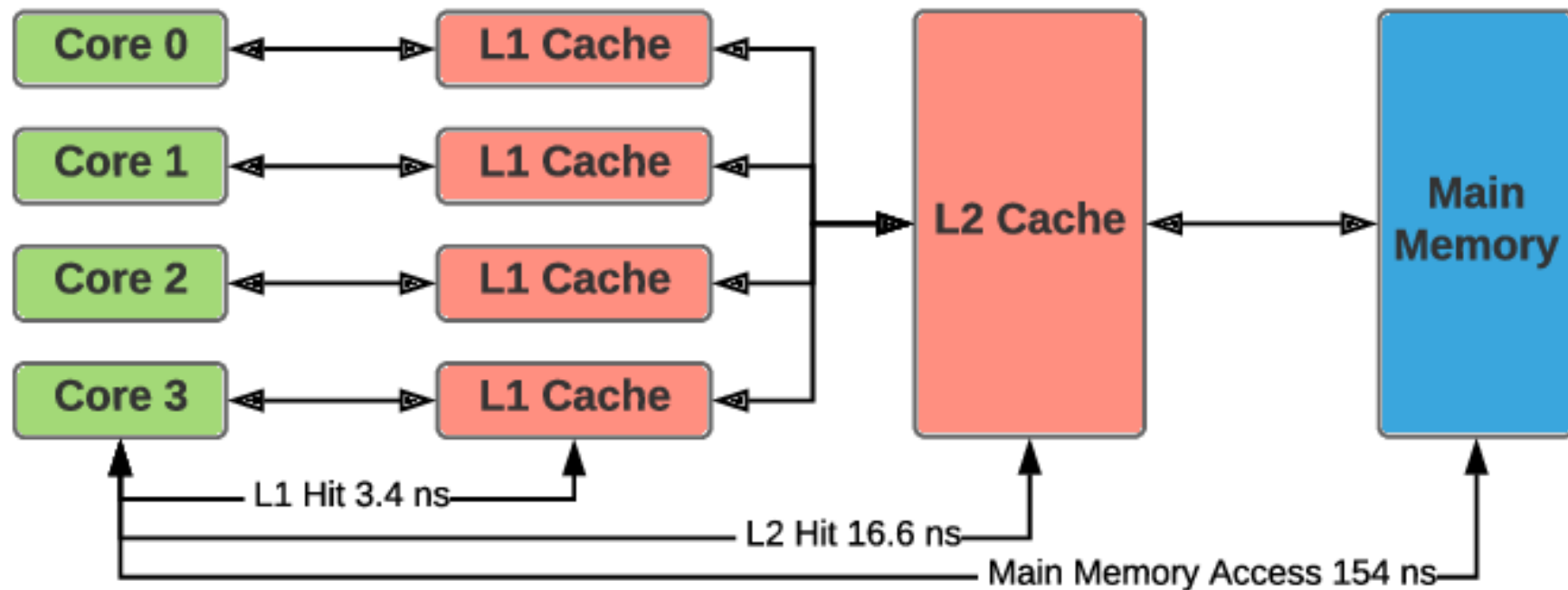
AMD EPYC 7003X Milan X CCD And 3D V Cache Architecture



ARM Cortex-A72 Cache Hierarchy



ARM Cortex A53 Cache Hierarchy



Different Types of Storage Elements

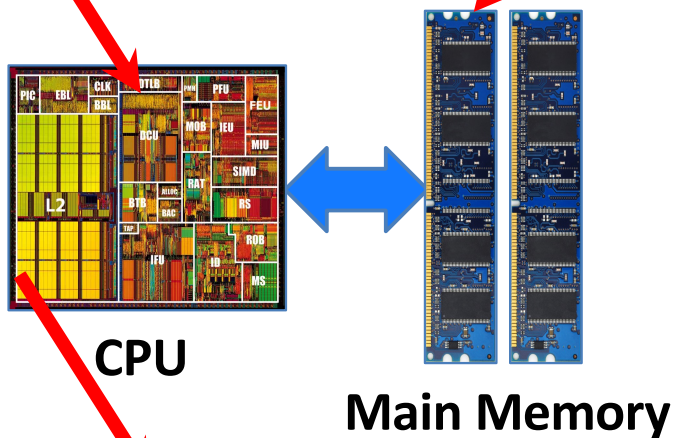
Diff. Storage Elements – Motivation

- **Flip-flops, registers, and flip-flop-based register files make up synchronous sequential circuits**
 - They process information based on current and past combinations of inputs
 - They have arithmetic circuits or logic elements to process inputs and history *stored as state*
 - Processing happens during a clock cycle, while state changes happen at the edge of the clock (**synchronous**)
- **Sometimes, we need to store lots of information**
 - Computer main memory, USB drives, flash drives, disks, SSDs
 - We can build cheaper storage elements that store information asynchronously – and processing happens somewhere else synchronously

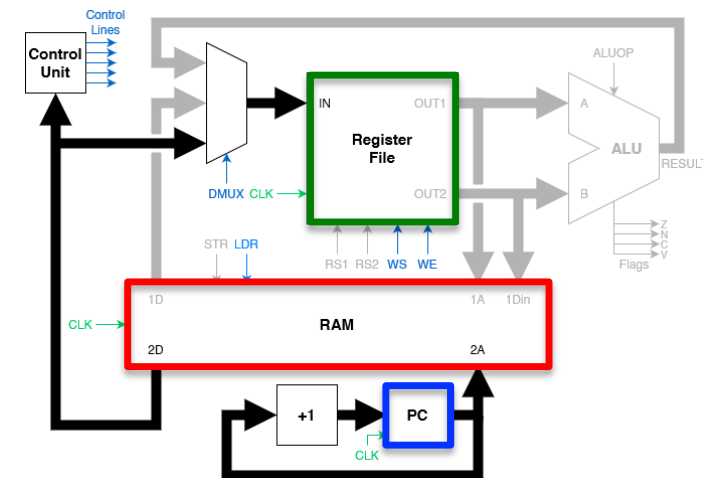
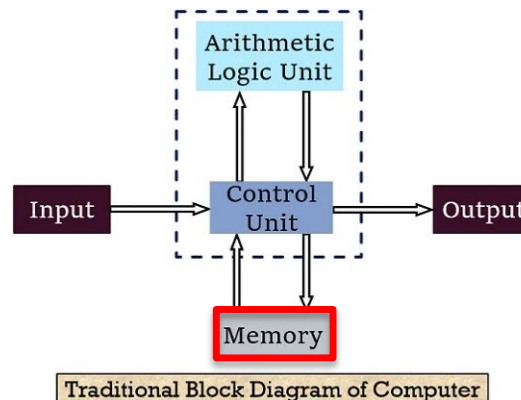
Example

Composed using ALUs, registers, muxes, and decoder to process one instruction every cycle

Just rows and rows of storage elements, and no cycle-by-cycle processing
Just read and write words



- On-chip fast memory called a L1/L2/L3 cache
- Subset of recently read data from main memory



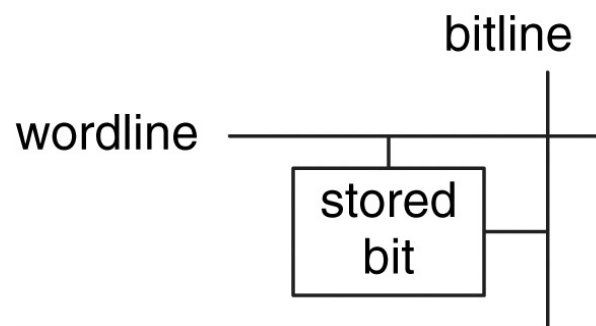
Storage Elements

- **Flip-flops**
 - Very fast (data is always available at the output)
 - Very expensive (one bit costs tens of transistors)
- **Static RAM (SRAM)**
 - Faster than DRAM
 - Expensive (one bit costs 6 transistors)
- **Dynamic RAM (DRAM)**
 - Slower than latches, flip-flops, and SRAM
 - Reading destroys content, requiring a refresh operation to recharge the capacitor
 - It needs a special process for manufacturing
 - Cheap (one bit costs only one transistor plus one transistor)

Section 5.5 of H&H

Generalized Storage Element

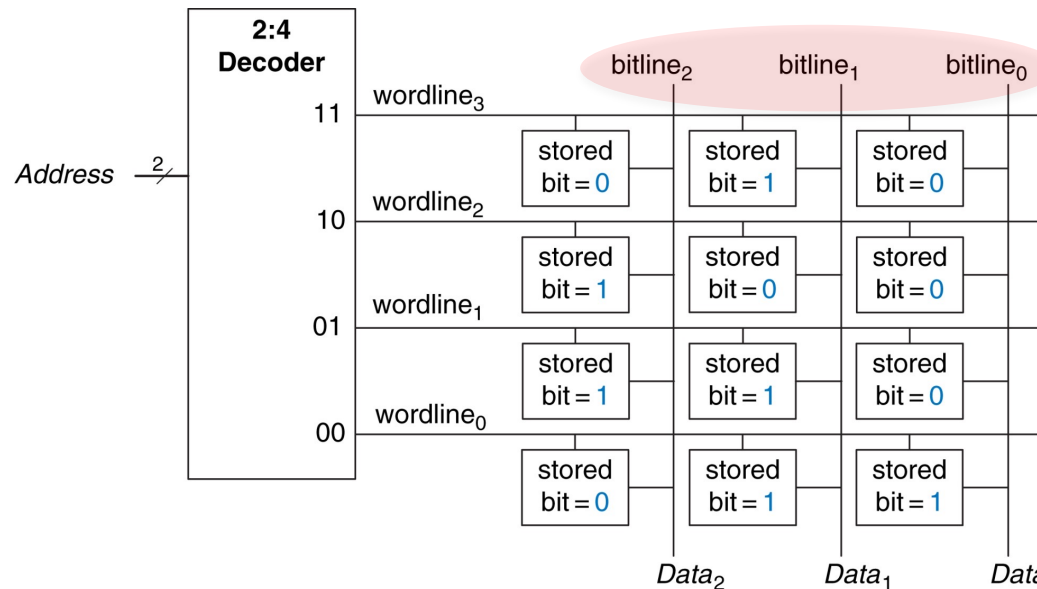
- Stores **one** bit (**bit cell**)
 - **Wordline enables** (**selects**) the storage element or bit cell
 - Bit line is used to **read** the stored bit or **write** a new bit



- Why is it called **word line**?
 - Word line **addresses** an entire (**unique**) word in a row of words
 - We **read** from and **write** to an entire row

Generalized Memory Organization

- A **decoder** to decode the address
- A **multiplexer** to select an output (**not shown below**)
 - Can also use **tri-state buffers** instead of AND/OR multiplexer



- Storage element can be of any "type"
- **Address/Data** makes up a **single read/write port**

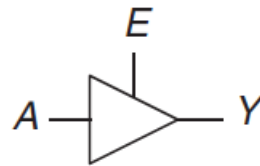
Section 5.5 of H&H

rightmost line is numbered 0

Recall a Tri-State Buffer

- A tri-state buffer enables gating of different signals onto a wire

Tristate
Buffer



<i>E</i>	<i>A</i>	<i>Y</i>
0	0	Z
0	1	Z
1	0	0
1	1	1

**A tri-state buffer
acts like a switch**

Figure 2.40 Tristate buffer

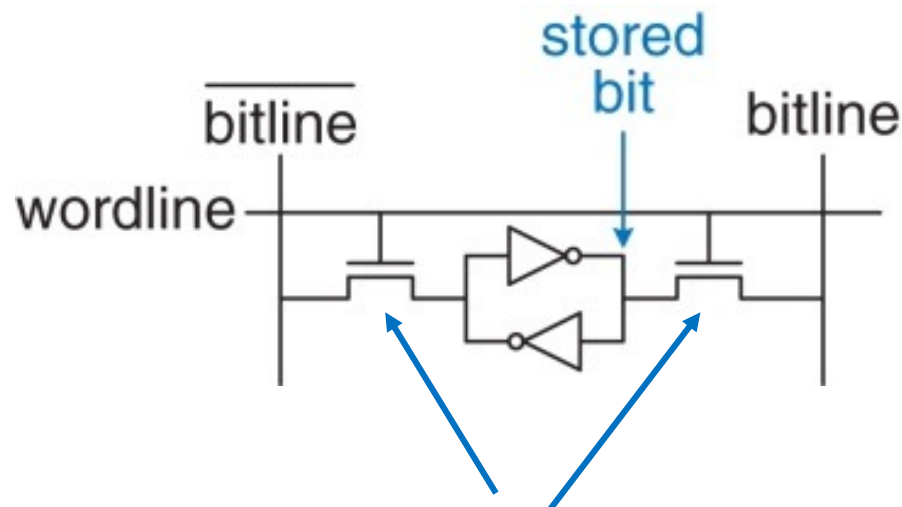
- When *E* is LOW, output is a floating signal (Z)
- **Floating:** Signal not driven by any circuit (open circuit, floating wire)

Read/Write Procedure in Detail

- **READ**
 - A **wordline** is asserted, and the corresponding row of bits cells drives the bitlines **HIGH** or **LOW**
- **WRITE**
 - The bitlines are driven **HIGH** or **LOW** first and then a **wordline** is **asserted**, allowing the **bitlines** values to be stored in that row of bit cells
- **Bitlines are used for both reading the stored bit and writing a new bit (for example, using a pair of tristate buffers)**

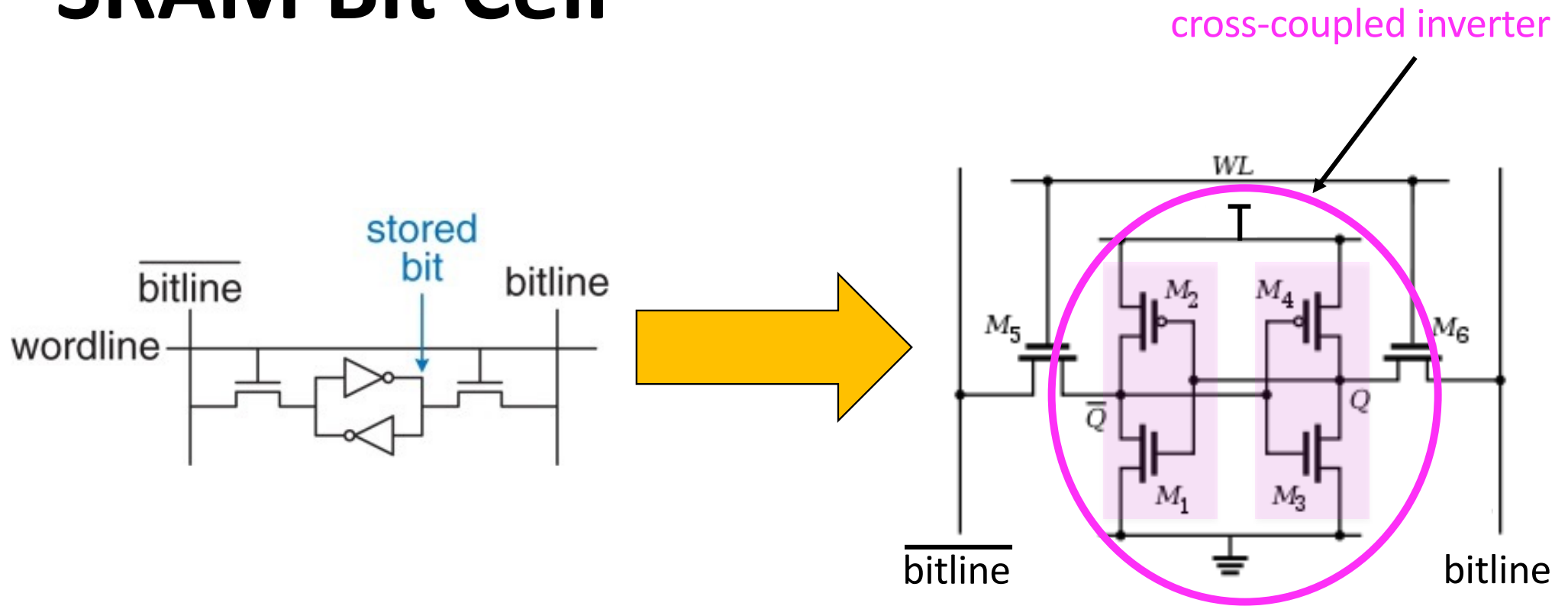
SRAM Bit Cell

- **SRAM**: Static Random Access Memory
 - Data bit is stored in a cross-coupled inverter
 - Each cell has two outputs: bitline and **bitline** and **bitline**



- When the wordline is asserted, both nMOS transistors are turned **ON**, and data values are **transferred** *to or from* the **bitlines**

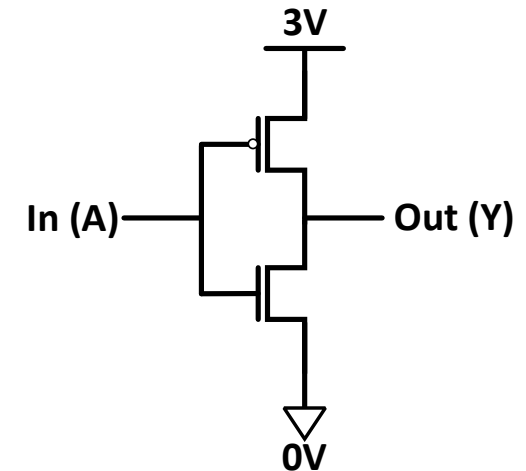
SRAM Bit Cell



source: <https://electronics.stackexchange.com/questions/162466/how-do-the-access-transistors-in-an-sram-cell-work>

Recall: MOS NOT Gate (Inverter)

- We have seen a NOT gate at the transistor level
 - If A = **0V** then Y = **3V**
 - If A = **3V** then Y = **0V**
- Interpretation of voltage levels
 - Interpret **0V** as logical (binary) **0** value
 - Interpret **3V** as logical (binary) **1** value



A	P	N	Y
0	ON	OFF	1
1	OFF	ON	0

$$Y = \bar{A}$$

Random Access Memory (RAM)

- **RAM**: Random Access Memory
 - Each byte has an address (byte-addressable)
 - Any data word (*consisting of multiple bytes*) can be accessed independently of the other
 - Any data word is accessed with the same delay as any other

Another Device for Storing Information

- What storage medium is this?



- A length of magnetic tape in a plastic enclosure with one or two reels for controlling the motion of the tape
- **Cassette used in tape recorders:** nearby data is accessed more quickly than faraway data
- **Inherently sequential device:** they must be rewound and read from the start to load data

Yet Another Device for Storing Information

- What storage medium is this?



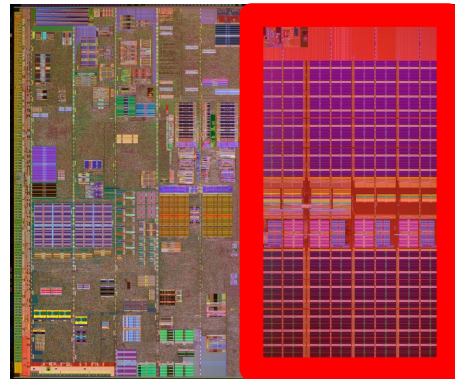
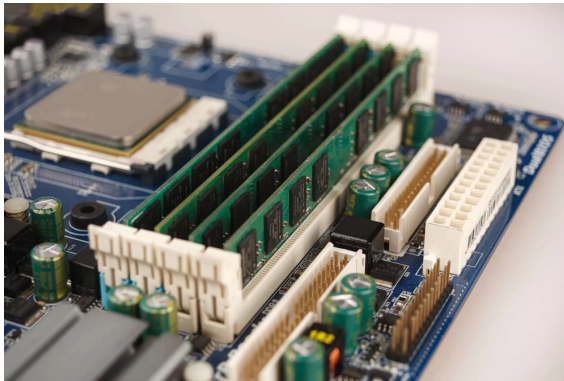
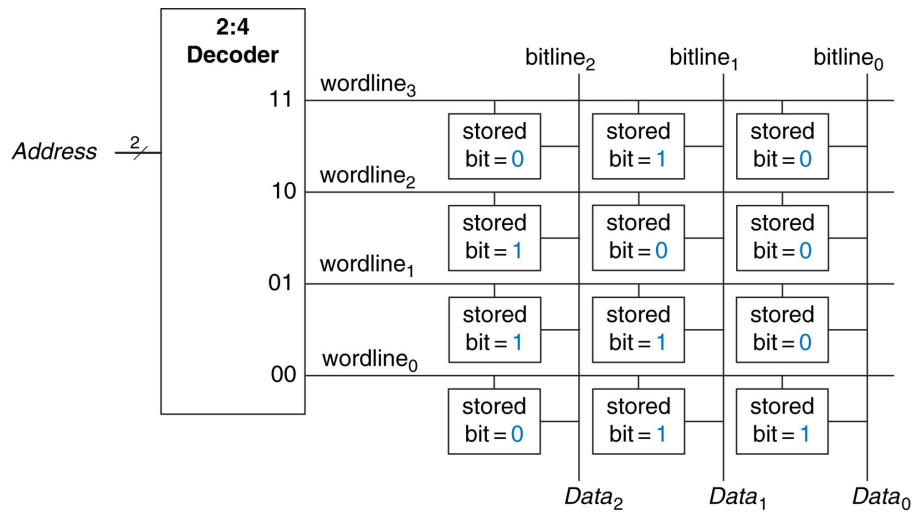
- A rotational platter with magnetic coating and arm assembly inside a casing
- **Hard disk used in all computers:** nearby data is accessed more quickly than faraway data
- **Inherently sequential device:** a disk must rotate under an arm for data to be read

Random vs. Sequential access storage media

These

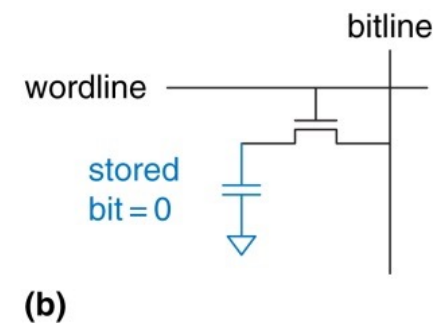
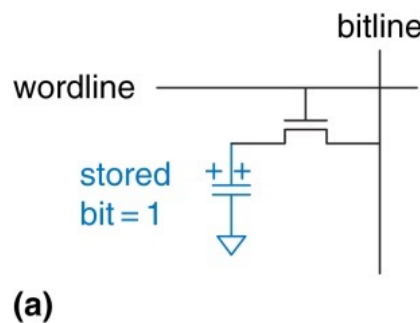
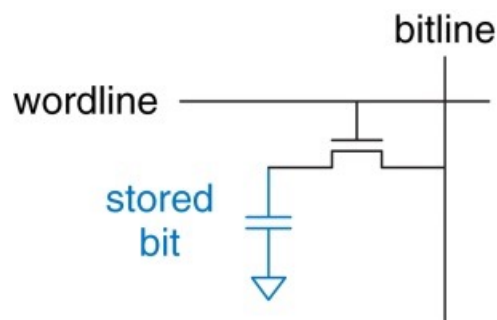
vs.

These



DRAM Bit Cell

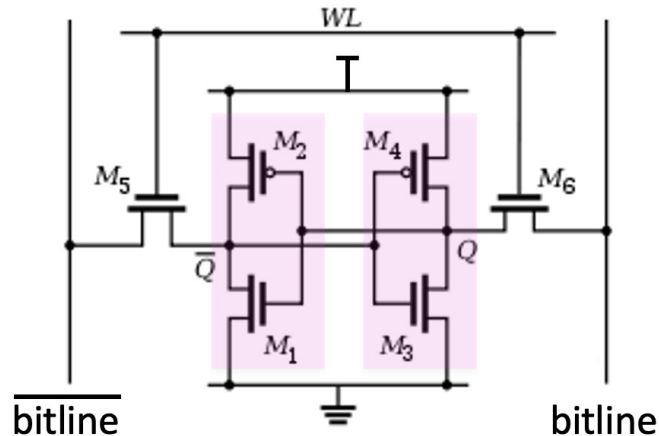
- **DRAM**: **D**ynamic **R**andom **A**ccess **M**emory
 - Stores a **bit** as the presence or absence of a **charge** on a capacitor
 - The **nMOS** transistor behaves as a **switch** that either connects or disconnects the capacitor from the **bitline**



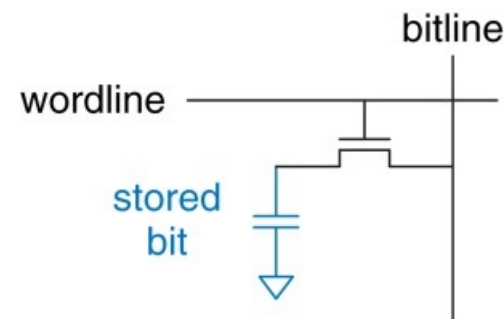
- When the wordline is **asserted**, the **nMOS transistor** turns **ON**, and the stored bit value **transfers to or from the bitline**

Static vs. Dynamic

- Recall the **SRAM** cell: The **nMOS transistors** (M5 and M6) are driven by cross-coupled inverter connected to **supply voltage**
- DRAM**: The capacitor node is not actively driven **HIGH** or **LOW** by a transistor tied to **supply voltage**



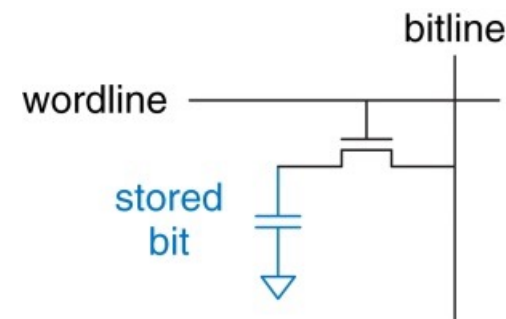
SRAM Cell



DRAM Cell

DRAM Reads and Writes

- **Read:** Upon a **read**, data values are **transferred** from the capacitor to the bitline
 - **Reading destroys** the bit value stored on the capacitor, so the data word must be **restored** (**rewritten**) after each **read**
 - **Refresh:** Even when DRAM is not **read**, the contents must be **refreshed** (read and rewritten) every **few milliseconds**, because the charge on the capacitor gradually **leaks** away



- **Write:** Upon a **write**, data values are **transferred** from the bitline to the capacitor

DRAM Refresh

- **Refresh:** Even when DRAM is not **read**, the contents must be **refreshed** (read and rewritten) every **few milliseconds** because *the charge on the capacitor* gradually **leaks** away
- **Refreshing consumes** extra energy
- **Refreshing** requires costly circuitry
 - It is a **critical drawback** of DRAM technology, especially at large capacities (many gigabytes, for example)

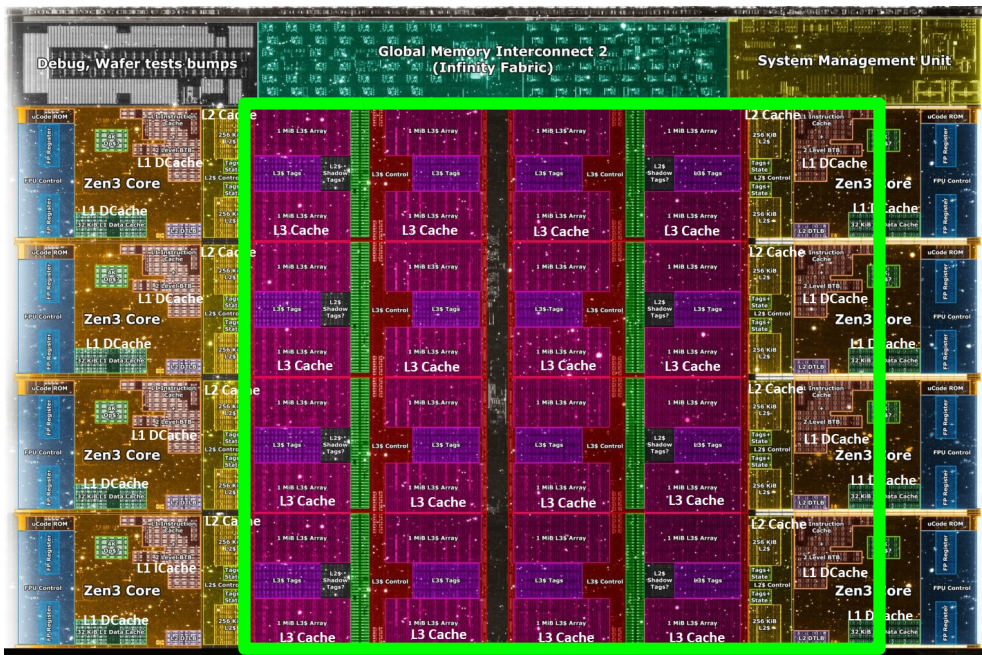
Memory Comparison

- **Latches and flip-flops (~20 transistors)**
 - **Very fast**
 - **Very expensive** (one bit costs tens of transistors)
- **Static RAM (SRAM) (6 transistors)**
 - **Relatively fast**
 - **Expensive** (one bit costs 6 transistors)
- **Dynamic RAM (DRAM) (1 transistor)**
 - **Slower than latches, flip-flops, and SRAM**
 - **Reading destroys content**, requiring a **refresh** operation to recharge the capacitor
 - **It needs a special process for manufacturing**
 - **Cheap** (one bit **costs only one transistor plus one transistor**)

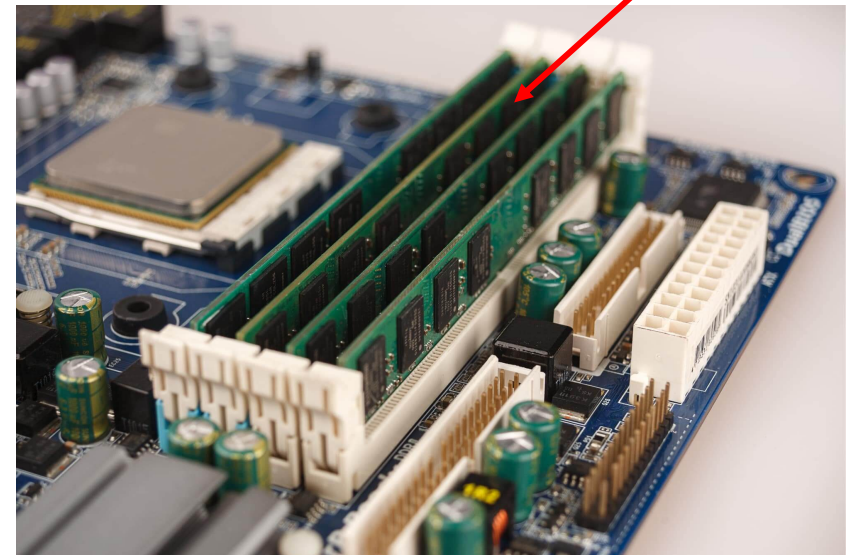
Section 5.5 of H&H

Memory Hierarchy

Inside the CPU chip: SRAM and flip-flops



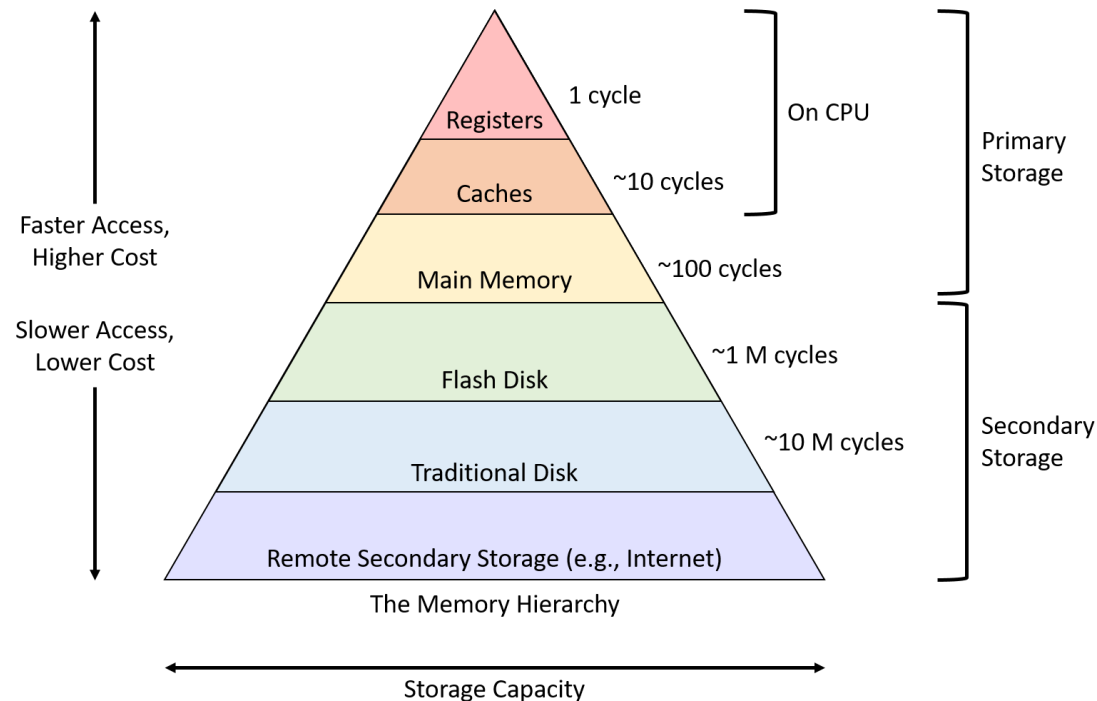
Outside the CPU chip: DRAM



DRAM Stick or DIMM

Memory Hierarchy

- Large memories use **SRAM** and **DRAM**
- Registers inside the **CPU** use **flip-flops**



- Flip-flops are used in **synchronous sequential circuits**